ADVANCED PATH | WORKBOOK

Building Custom AI Solutions

Print it, write in it, keep it. Your lab notes, debugging log and reflections for all 20 days.

About me

Name: _____ Phone: _____

What I do now: _____

Why I joined this course:

My three goals for these four weeks:

Tools and accounts checklist

Tool	Installed / created	2-step on	Notes
Python 3.10+ and VS Code			
Git and GitHub account			
LLM provider API key (in .env)			
Google Colab			
Hosting account (Render / HF)			

How to use this workbook

Each day has two pages. Page one is for the lab: write your prompts, record results and notes. Page two is for review: questions, reflection and homework. Use a pen. Write messy notes: this is your thinking space.

Progress tracker

Week	Deliverable	Done
Week 1	API toolkit repo with 5 scripts, structured extraction project	
Week 2	Deployed-ready RAG app with UI and evaluation report	
Week 3	Working agent with 3 tools, safety checks and logs	
Week 4	Deployed capstone, repo, technical report and demo	

Week 1: Python, APIs and LLM Foundations

Goal: Be fluent in calling models programmatically and controlling their output.

Deliverable: API toolkit repo with 5 scripts, structured extraction project

My target for this week:

Weekly checklist

[ ] Day 1: Dev environment and Python refresher

[ ] Day 2: First LLM API calls

[ ] Day 3: Structured outputs and extraction

[ ] Day 4: Embeddings and semantic search

[ ] Day 5: Lab day: Mini project sprint

At the end of the week I will show:

Day 1: Dev environment and Python refresher

Objectives

[ ] Set up a clean Python project with Git, a virtual environment and a safe .env file

[ ] Call a public REST API and parse the JSON response

[ ] Explain tokens, context windows, temperature and cost

[ ] Push your first repository to GitHub

Warm-up: one thing I tried since last class

Lab steps: tick each step and write notes

[ ] **Step 1.** Install Python, VS Code and Git; confirm with python --version and git --version

[ ] **Step 2.** Create the project folder, run git init and create and activate a virtual environment

[ ] **Step 3.** Create .gitignore (with .env and the venv folder) and a .env.example

[ ] **Step 4.** Write fetch.py that calls a public JSON API with requests, sets a timeout and prints three fields

[ ] **Step 5.** Estimate the token cost of five of your own prompts using the provider's pricing page

[ ] **Step 6.** Create a GitHub repository, push your first commit and share the link with the trainer

Debugging log: errors I hit today

Error message	My guess	What fixed it

Code notes (what each part does, in my own words)

Day 1 review

Check your understanding

Answer in your own words, then compare with your notes and code.

Q1. Why use a virtual environment?

Q2. Where should an API key be stored?

Q3. What does status code 429 mean?

Q4. What is a token?

Q5. When is low temperature useful?

Words I learned today (write the meaning in your own words)

Word	My meaning
API	
Virtual environment	
Token	
Context window	

Reflection

Today I am proud that:

The hardest part was:

One thing I will use this week:

Homework / challenge

Fix 5 broken scripts in the 'debug kit'.

Done? [ ] How it went:

Day 2: First LLM API calls

Objectives

[ ] Make a chat completion call from Python and read the reply

[ ] Control behaviour with system prompts and parameters

[ ] Handle errors, retries and rate limits

[ ] Build a command-line assistant with memory and a cost counter

Warm-up: one thing I tried since last class

Lab steps: tick each step and write notes

[ ] **Step 1.** Install your provider's SDK and load the key from .env

[ ] **Step 2.** Send one message and print only the reply text

[ ] **Step 3.** Add a system prompt from prompts/assistant_v1.txt and compare behaviour with and without it

[ ] **Step 4.** Keep a history list and trim it to the last 10 messages

[ ] **Step 5.** Add retry with backoff for timeouts and rate limits, and a friendly error message

[ ] **Step 6.** Add a daily cost counter using token counts from each response and print it on exit

Debugging log: errors I hit today

Error message	My guess	What fixed it

Code notes (what each part does, in my own words)

Day 2 review

Check your understanding

Answer in your own words, then compare with your notes and code.

Q1. Why must you resend history?

Q2. What is exponential backoff?

Q3. Which errors should you not retry?

Q4. How can you limit context cost?

Q5. Where do stable instructions belong?

Words I learned today (write the meaning in your own words)

Word	My meaning
System prompt	
Stateless	
Backoff	
SDK	

Reflection

Today I am proud that:

The hardest part was:

One thing I will use this week:

Homework / challenge

Add logging and retries to your CLI.

Done? [ ] How it went:

Day 3: Structured outputs and extraction

Objectives

[ ] Get reliable structured output from a model

[ ] Validate output with Pydantic

[ ] Build an extractor that turns messy documents into a clean CSV

[ ] Measure accuracy against hand-labelled samples

Warm-up: one thing I tried since last class

Lab steps: tick each step and write notes

[ ] **Step 1.** Write 30 synthetic receipts or invoices with deliberately messy formatting

[ ] **Step 2.** Define a Pydantic model: vendor, date, items, total, currency

[ ] **Step 3.** Write the extraction prompt with schema, rules and two examples

[ ] **Step 4.** Run all documents, validate each, retry invalid ones once with the error message

[ ] **Step 5.** Write results to a CSV and log documents that still fail

[ ] **Step 6.** Hand-label 20, compute per-field accuracy and write down the top three failure causes

Debugging log: errors I hit today

Error message	My guess	What fixed it

Code notes (what each part does, in my own words)

Day 3 review

Check your understanding

Answer in your own words, then compare with your notes and code.

Q1. Why validate model output?

Q2. What does Pydantic do?

Q3. Why prefer null to guessing?

Q4. Why measure accuracy per field?

Q5. What should you keep from your labelled set?

Words I learned today (write the meaning in your own words)

Word	My meaning
Schema	
Validation	
Few-shot	
Regression test	

Reflection

Today I am proud that:

The hardest part was:

One thing I will use this week:

Homework / challenge

Measure accuracy on 20 hand-labelled samples.

Done? [ ] How it went:

Day 4: Embeddings and semantic search

Objectives

[ ] Explain embeddings and cosine similarity

[ ] Chunk documents sensibly with overlap and metadata

[ ] Store and search vectors with Chroma or FAISS

[ ] Compare chunk sizes by measuring search quality

Warm-up: one thing I tried since last class

Lab steps: tick each step and write notes

[ ] **Step 1.** Choose a public document set of about 100 pages (for example county service guides or public statute extracts)

[ ] **Step 2.** Extract text from each file and print the first 500 characters to check quality

[ ] **Step 3.** Split into chunks with overlap and attach source and page metadata

[ ] **Step 4.** Create a Chroma collection, add all chunks and persist it

[ ] **Step 5.** Write search(query, k) that prints the top results with their sources

[ ] **Step 6.** Write 20 questions with expected sources, compute hit rate at 3 for two chunk sizes

Debugging log: errors I hit today

Error message	My guess	What fixed it

Code notes (what each part does, in my own words)

Day 4 review

Check your understanding

Answer in your own words, then compare with your notes and code.

Q1. What is an embedding?

Q2. What does cosine similarity measure?

Q3. Why use overlap?

Q4. Why store metadata?

Q5. What is hit rate at k?

Words I learned today (write the meaning in your own words)

Word	My meaning
Embedding	
Chunk	
Vector store	
Top-k	

Reflection

Today I am proud that:

The hardest part was:

One thing I will use this week:

Homework / challenge

Try 3 different chunk sizes and compare.

Done? [ ] How it went:

Day 5: Lab day: Mini project sprint

Objectives

[ ] Plan and build a small AI tool in one day

[ ] Practise pair programming and code review

[ ] Document your tool in a README

[ ] Present a working demo to peers

Warm-up: one thing I tried since last class

Lab steps: tick each step and write notes

[ ] **Step 1.** Choose the tool and write the user sentence, success sentence and three must-have features

[ ] **Step 2.** Create the repo with .gitignore, .env.example and README skeleton

[ ] **Step 3.** Build the first feature end-to-end, commit, then swap driver and navigator

[ ] **Step 4.** Add error handling and one automated test (or a small test script)

[ ] **Step 5.** Pair review: use the four-question checklist and exchange comments

[ ] **Step 6.** Finish the README and give a five-minute demo including one honest failure

Debugging log: errors I hit today

Error message	My guess	What fixed it

Code notes (what each part does, in my own words)

Day 5 review

Check your understanding

Answer in your own words, then compare with your notes and code.

Q1. What roles are in pair programming?

Q2. What four questions guide a code review?

Q3. What belongs in a README?

Q4. Why show a failure in a demo?

Q5. Why time-box?

Words I learned today (write the meaning in your own words)

Word	My meaning
Driver	
Navigator	
Code review	
README	

Reflection

Today I am proud that:

The hardest part was:

One thing I will use this week:

Homework / challenge

Refactor based on review.

Done? [ ] How it went:

Week 2: RAG and AI Applications

Goal: Build applications that answer from real documents and look professional.

Deliverable: Deployed-ready RAG app with UI and evaluation report

My target for this week:

Weekly checklist

[ ] Day 6: RAG architecture

[ ] Day 7: Better retrieval

[ ] Day 8: User interfaces

[ ] Day 9: Evaluation and quality

[ ] Day 10: Lab day: Domain RAG project

At the end of the week I will show:

Day 6: RAG architecture

Objectives

[ ] Describe the retrieve, augment, generate flow of RAG

[ ] Build a basic RAG pipeline with source citations

[ ] Write prompts that keep answers grounded in documents

[ ] Know when RAG, fine-tuning or plain prompting is the right choice

Warm-up: one thing I tried since last class

Lab steps: tick each step and write notes

[ ] **Step 1.** Reuse last week's index and add one more document set

[ ] **Step 2.** Write retrieve(question) returning the top 5 chunks with ids and sources

[ ] **Step 3.** Write the augmented prompt template with numbered context and the grounding rules

[ ] **Step 4.** Call the model and print the answer with the cited sources underneath

[ ] **Step 5.** Ask 20 test questions: ten answerable, five unanswerable, five tricky; record the results

[ ] **Step 6.** Write the 20 questions with expected answers into questions.json for tomorrow

Debugging log: errors I hit today

Error message	My guess	What fixed it

Code notes (what each part does, in my own words)

Day 6 review

Check your understanding

Answer in your own words, then compare with your notes and code.

Q1. What are the three steps of RAG?

Q2. Why cite chunk ids?

Q3. When is RAG better than fine-tuning?

Q4. What should the model say when context lacks the answer?

Q5. Where do most RAG bugs hide?

Words I learned today (write the meaning in your own words)

Word	My meaning
RAG	
Grounding	
Fine-tuning	
Hallucination	

Reflection

Today I am proud that:

The hardest part was:

One thing I will use this week:

Homework / challenge

Write 20 test questions with expected answers.

Done? [ ] How it went:

Day 7: Better retrieval

Objectives

[ ] Improve retrieval with query rewriting and metadata filters

[ ] Understand hybrid search and re-ranking

[ ] Handle tables and scanned documents with OCR

[ ] Measure the effect of each improvement

Warm-up: one thing I tried since last class

Lab steps: tick each step and write notes

[ ] **Step 1.** Load your 20-question test set from yesterday and record the baseline hit rate at 5

[ ] **Step 2.** Implement query rewriting using the last three messages; re-measure

[ ] **Step 3.** Add a metadata filter (document type or year) and measure again

[ ] **Step 4.** Add keyword scoring and merge with semantic results; measure

[ ] **Step 5.** Run OCR on two scanned pages and inspect the quality

[ ] **Step 6.** Write a one-page comparison table: baseline vs each improvement, with a decision on what to keep

Debugging log: errors I hit today

Error message	My guess	What fixed it

Code notes (what each part does, in my own words)

Day 7 review

Check your understanding

Answer in your own words, then compare with your notes and code.

Q1. Why rewrite queries?

Q2. What is hybrid search?

Q3. What does a re-ranker do?

Q4. Why keep original page links?

Q5. How do you know an improvement helped?

Words I learned today (write the meaning in your own words)

Word	My meaning
Re-ranking	
BM25	
OCR	
Metadata filter	

Reflection

Today I am proud that:

The hardest part was:

One thing I will use this week:

Homework / challenge

Log retrieval failures and categorize them.

Done? [ ] How it went:

Day 8: User interfaces

Objectives

[ ] Build a chat interface with Streamlit or Gradio

[ ] Show sources, loading states and feedback buttons

[ ] Design for honesty: 'I do not know' and limits

[ ] Test usability with three real users

Warm-up: one thing I tried since last class

Lab steps: tick each step and write notes

[ ] **Step 1.** Create app.py with st.chat_input and st.chat_message and keep history in session_state

[ ] **Step 2.** Cache the index loading with st.cache_resource

[ ] **Step 3.** Show sources in an expander under each answer

[ ] **Step 4.** Add thumbs up/down buttons that append the question, answer and rating to feedback.csv

[ ] **Step 5.** Add an 'I could not find this' state with a suggestion to rephrase

[ ] **Step 6.** Run it on your phone over a mobile connection and have three people test it while you watch silently

Debugging log: errors I hit today

Error message	My guess	What fixed it

Code notes (what each part does, in my own words)

Day 8 review

Check your understanding

Answer in your own words, then compare with your notes and code.

Q1. Why use session_state in Streamlit?

Q2. What should an answer display besides text?

Q3. What should happen when nothing relevant is found?

Q4. Why stream answers?

Q5. Why test on a real phone?

Words I learned today (write the meaning in your own words)

Word	My meaning
Session state	
Caching	
UX	
Streaming	

Reflection

Today I am proud that:

The hardest part was:

One thing I will use this week:

Homework / challenge

Get 3 users to test it and record notes.

Done? [ ] How it went:

Day 9: Evaluation and quality

Objectives

[ ] Build an evaluation script with a test set and scores

[ ] Measure groundedness, relevance and refusal behaviour

[ ] Understand the limits of LLM-as-judge

[ ] Set up regression tests that run before every change

Warm-up: one thing I tried since last class

Lab steps: tick each step and write notes

[ ] **Step 1.** Expand questions.json to 30 items with fields id, question, expected_source, type and expected_answer

[ ] **Step 2.** Write eval.py that runs every question through your pipeline and stores answer, sources and timing

[ ] **Step 3.** Add automatic checks: right source retrieved, citation present, refusal on unanswerable questions

[ ] **Step 4.** Add an LLM judge with a 4-question yes/no rubric and spot-check 10 judgements manually

[ ] **Step 5.** Print a summary table of metrics and save results with date and git commit

[ ] **Step 6.** Fix the five worst failures, re-run and show the before and after table

Debugging log: errors I hit today

Error message	My guess	What fixed it

Code notes (what each part does, in my own words)

Day 9 review

Check your understanding

Answer in your own words, then compare with your notes and code.

Q1. Why evaluate retrieval and generation separately?

Q2. What is groundedness?

Q3. Name two weaknesses of LLM judges.

Q4. Why hold back some test questions?

Q5. Why record commit and metrics?

Words I learned today (write the meaning in your own words)

Word	My meaning
Groundedness	
Overfitting	
Rubric	
Latency	

Reflection

Today I am proud that:

The hardest part was:

One thing I will use this week:

Homework / challenge

Improve the worst 5 failures.

Done? [ ] How it went:

Day 10: Lab day: Domain RAG project

Objectives

[ ] Build a domain-specific RAG prototype on a real use case

[ ] Handle data permissions and privacy correctly

[ ] Write an evaluation report for your prototype

[ ] Demonstrate the prototype and explain its limits

Warm-up: one thing I tried since last class

Lab steps: tick each step and write notes

[ ] **Step 1.** Choose the domain and write the user sentence, the cost-of-error sentence and the data permissions note

[ ] **Step 2.** Ingest at least 20 documents with metadata including type, date and access label

[ ] **Step 3.** Run the full pipeline and your 30-question evaluation; record the baseline

[ ] **Step 4.** Improve the weakest metric using one change at a time, measuring each

[ ] **Step 5.** Complete the interface with sources, feedback and a coverage note

[ ] **Step 6.** Write the one-page evaluation report and demo to peers, including one failure

Debugging log: errors I hit today

Error message	My guess	What fixed it

Code notes (what each part does, in my own words)

Day 10 review

Check your understanding

Answer in your own words, then compare with your notes and code.

Q1. Why ask the cost of a wrong answer?

Q2. How can you enforce access rights in RAG?

Q3. What should a report include besides success?

Q4. Why tag working versions?

Q5. Name one data protection principle.

Words I learned today (write the meaning in your own words)

Word	My meaning
Access label	
Data minimisation	
Prototype	
Baseline	

Reflection

Today I am proud that:

The hardest part was:

One thing I will use this week:

Homework / challenge

Write a one-page README with limits.

Done? [ ] How it went:

Week 3: Agents, Tools and Integrations

Goal: Create AI systems that take actions through tools, safely and reliably.

Deliverable: Working agent with 3 tools, safety checks and logs

My target for this week:

Weekly checklist

[ ] Day 11: Tool use and function calling

[ ] Day 12: Agent design patterns

[ ] Day 13: Real-world integrations

[ ] Day 14: Safety, security and abuse prevention

[ ] Day 15: Lab day: Agent project sprint

At the end of the week I will show:

Day 11: Tool use and function calling

Objectives

[ ] Define tools with schemas and let a model call them

[ ] Run the tool-call loop with a step limit

[ ] Log every tool call for debugging

[ ] Recognise common agent failure modes

Warm-up: one thing I tried since last class

Lab steps: tick each step and write notes

[ ] **Step 1.** Define three tools: calculator, lookup_record (from a small local database) and get_time, each with a clear schema

[ ] **Step 2.** Implement the functions with argument validation

[ ] **Step 3.** Write the agent loop with a maximum of 6 steps

[ ] **Step 4.** Log every tool call, arguments, result and elapsed time to agent.log

[ ] **Step 5.** Test ten tasks including some requiring two tools and some requiring none

[ ] **Step 6.** Break it on purpose: ambiguous requests, missing data, a tool that returns an error

Debugging log: errors I hit today

Error message	My guess	What fixed it

Code notes (what each part does, in my own words)

Day 11 review

Check your understanding

Answer in your own words, then compare with your notes and code.

Q1. Who executes the tool?

Q2. What is in a tool definition?

Q3. Why set a maximum number of steps?

Q4. How can you reduce invented arguments?

Q5. Why log every tool call?

Words I learned today (write the meaning in your own words)

Word	My meaning
Tool calling	
Agent	
Schema	
Step limit	

Reflection

Today I am proud that:

The hardest part was:

One thing I will use this week:

Homework / challenge

Add 2 new tools.

Done? [ ] How it went:

Day 12: Agent design patterns

Objectives

[ ] Compare common agent design patterns

[ ] Build a research agent that plans, searches, drafts and cites

[ ] Manage state, memory and long-running tasks

[ ] Set and enforce cost and latency budgets

Warm-up: one thing I tried since last class

Lab steps: tick each step and write notes

[ ] **Step 1.** Choose a research question (for example 'summarise the main options for a small farmer to irrigate half an acre') and your document or web-search tool

[ ] **Step 2.** Build version A as a fixed workflow: search, read, draft, cite

[ ] **Step 3.** Build version B as an agent with tools and a plan

[ ] **Step 4.** Add a reflection step that checks citations against sources

[ ] **Step 5.** Add budgets for steps, tokens and time and a graceful stop

[ ] **Step 6.** Compare A and B on ten tasks for quality, cost and time; write your decision

Debugging log: errors I hit today

Error message	My guess	What fixed it

Code notes (what each part does, in my own words)

Day 12 review

Check your understanding

Answer in your own words, then compare with your notes and code.

Q1. What is the difference between a workflow and an agent?

Q2. Name two agent patterns.

Q3. Where should long-task state be stored?

Q4. Why use different models for different steps?

Q5. When should a human approve?

Words I learned today (write the meaning in your own words)

Word	My meaning
Workflow	
Routing	
Reflection	
Human in the loop	

Reflection

Today I am proud that:

The hardest part was:

One thing I will use this week:

Homework / challenge

Compare 2 designs on 10 tasks.

Done? [ ] How it went:

Day 13: Real-world integrations

Objectives

[ ] Integrate an AI assistant with Google Sheets and messaging webhooks

[ ] Understand the flow of a mobile payment request in a sandbox

[ ] Handle callbacks safely and idempotently

[ ] Draw a system architecture diagram

Warm-up: one thing I tried since last class

Lab steps: tick each step and write notes

[ ] **Step 1.** Set up a Google Sheet with a product catalogue and an Orders tab, and a service account or API access

[ ] **Step 2.** Write a small FastAPI or Flask app with a /webhook endpoint that you can test locally with sample JSON

[ ] **Step 3.** Use the assistant to extract the order from a customer message and validate it against the catalogue

[ ] **Step 4.** Write the order to Sheets with a unique event id and skip duplicates

[ ] **Step 5.** Following the Daraja sandbox docs, request a test payment and handle the callback in /payment-callback

[ ] **Step 6.** Draw the architecture diagram and mark every place where secrets or personal data appear

Debugging log: errors I hit today

Error message	My guess	What fixed it

Code notes (what each part does, in my own words)

Day 13 review

Check your understanding

Answer in your own words, then compare with your notes and code.

Q1. What is a webhook?

Q2. What is idempotency?

Q3. Why use a payment sandbox?

Q4. How do you handle duplicate events?

Q5. Why draw an architecture diagram?

Words I learned today (write the meaning in your own words)

Word	My meaning
Webhook	
Callback	
Idempotent	
Sandbox	

Reflection

Today I am proud that:

The hardest part was:

One thing I will use this week:

Homework / challenge

Draw the system architecture diagram.

Done? [ ] How it went:

Day 14: Safety, security and abuse prevention

Objectives

[ ] Identify prompt injection, data leakage and tool abuse risks

[ ] Apply least privilege to tools and credentials

[ ] Add rate limiting, authentication and logging

[ ] Red-team your own agent and document the fixes

Warm-up: one thing I tried since last class

Lab steps: tick each step and write notes

[ ] **Step 1.** List every tool and the permission each one has; reduce each to the minimum

[ ] **Step 2.** Write ten attack prompts: system prompt extraction, rule override, data theft, tool misuse, expensive loops

[ ] **Step 3.** Run the attacks against your agent and record which succeed

[ ] **Step 4.** Add fixes: input and output checks, tool argument validation, approval step and rate limit

[ ] **Step 5.** Add API key authentication and a simple per-user rate limit to your API

[ ] **Step 6.** Write a one-page security checklist for your capstone and add your attacks to the test set

Debugging log: errors I hit today

Error message	My guess	What fixed it

Code notes (what each part does, in my own words)

Day 14 review

Check your understanding

Answer in your own words, then compare with your notes and code.

Q1. What is prompt injection?

Q2. What does least privilege mean?

Q3. Name one defence against abuse of your API.

Q4. What is red-teaming?

Q5. Why add attacks to regression tests?

Words I learned today (write the meaning in your own words)

Word	My meaning
Prompt injection	
Least privilege	
Red team	
Rate limiting	

Reflection

Today I am proud that:

The hardest part was:

One thing I will use this week:

Homework / challenge

Write a security checklist for your project.

Done? [ ] How it went:

Day 15: Lab day: Agent project sprint

Objectives

[ ] Choose and scope a capstone project

[ ] Form a team with clear roles

[ ] Prototype an agent with at least two tools and an approval step

[ ] Write a one-page capstone proposal

Warm-up: one thing I tried since last class

Lab steps: tick each step and write notes

[ ] **Step 1.** Write the four scoping sentences and the list of features you will not build

[ ] **Step 2.** Choose team roles and set up the GitHub repository with branch rules

[ ] **Step 3.** Build the thinnest end-to-end prototype with at least two tools and an approval step

[ ] **Step 4.** Draw the architecture diagram and list the five biggest risks

[ ] **Step 5.** Write the one-page proposal and a 10-day plan

[ ] **Step 6.** Present a two-minute pitch and collect feedback from two peers

Debugging log: errors I hit today

Error message	My guess	What fixed it

Code notes (what each part does, in my own words)

Day 15 review

Check your understanding

Answer in your own words, then compare with your notes and code.

Q1. What four sentences scope a capstone?

Q2. What are the three team roles?

Q3. Why build a prototype first?

Q4. What goes in a proposal?

Q5. What should you do if scope is too big?

Words I learned today (write the meaning in your own words)

Word	My meaning
Scope	
Stand-up	
Pull request	
MVP	

Reflection

Today I am proud that:

The hardest part was:

One thing I will use this week:

Homework / challenge

Prepare a capstone proposal (1 page).

Done? [ ] How it went:

Week 4: Deployment and Capstone

Goal: Ship a secure, monitored AI application and present it professionally.

Deliverable: Deployed capstone, repo, technical report and demo

My target for this week:

Weekly checklist

[ ] Day 16: APIs and back-ends

[ ] Day 17: Deployment and monitoring

[ ] Day 18: Testing, polish and documentation

[ ] Day 19: Pitch and portfolio

[ ] Day 20: Demo Day and graduation

At the end of the week I will show:

Day 16: APIs and back-ends

Objectives

[ ] Wrap your AI logic in a FastAPI service with validated endpoints

[ ] Add authentication and request validation

[ ] Write unit tests for your endpoints

[ ] Containerise the service with Docker basics

Warm-up: one thing I tried since last class

Lab steps: tick each step and write notes

[ ] **Step 1.** Create api.py with FastAPI and Pydantic models for AskRequest and AskResponse

[ ] **Step 2.** Implement POST /ask that calls your pipeline and GET /health

[ ] **Step 3.** Add API key authentication from an environment variable and a simple rate limit

[ ] **Step 4.** Open /docs, try each endpoint and test the error cases

[ ] **Step 5.** Write at least five tests with the test client including unauthorised and invalid input

[ ] **Step 6.** Write a Dockerfile and a .dockerignore (including .env) and build the image locally if Docker is available

Debugging log: errors I hit today

Error message	My guess	What fixed it

Code notes (what each part does, in my own words)

Day 16 review

Check your understanding

Answer in your own words, then compare with your notes and code.

Q1. Why put AI logic behind an API?

Q2. What does FastAPI give automatically?

Q3. Which status code means unauthorised?

Q4. Why test with the test client?

Q5. What should never be in a Docker image?

Words I learned today (write the meaning in your own words)

Word	My meaning
Endpoint	
Docker image	
Authentication	
Constant-time comparison	

Reflection

Today I am proud that:

The hardest part was:

One thing I will use this week:

Homework / challenge

Add API key authentication.

Done? [ ] How it went:

Day 17: Deployment and monitoring

Objectives

[ ] Deploy the back-end and interface to a free hosting service

[ ] Set up logging, health checks and cost alerts

[ ] Understand environment variables in production

[ ] Estimate monthly cost and capacity

Warm-up: one thing I tried since last class

Lab steps: tick each step and write notes

[ ] **Step 1.** Push your latest code to GitHub with a clean README and requirements.txt

[ ] **Step 2.** Create the hosting service from the repository and set environment variables in its dashboard

[ ] **Step 3.** Deploy the API and the Streamlit interface; confirm /health and /docs work on the public URL

[ ] **Step 4.** Add structured logging with request id, latency and token counts, with personal data masked

[ ] **Step 5.** Write a cost estimate for 100 and 1,000 users and add a daily spending cap in code

[ ] **Step 6.** Share the live link with five testers and review the logs together

Debugging log: errors I hit today

Error message	My guess	What fixed it

Code notes (what each part does, in my own words)

Day 17 review

Check your understanding

Answer in your own words, then compare with your notes and code.

Q1. Where do production secrets go?

Q2. What should a log line contain?

Q3. Why have a health endpoint?

Q4. How do you estimate monthly cost?

Q5. Why add a hard spending cap?

Words I learned today (write the meaning in your own words)

Word	My meaning
Deployment	
Environment variable	
Monitoring	
Uptime	

Reflection

Today I am proud that:

The hardest part was:

One thing I will use this week:

Homework / challenge

Share your live link with 5 testers.

Done? [ ] How it went:

Day 18: Testing, polish and documentation

Objectives

[ ] Complete unit tests, an evaluation suite and a load test

[ ] Write a README, architecture diagram and limits section

[ ] Present a business case with costs and value

[ ] Fix the top issues from tester feedback

Warm-up: one thing I tried since last class

Lab steps: tick each step and write notes

[ ] **Step 1.** Write integration tests for /ask and /feedback and a small load test with 20 simultaneous requests

[ ] **Step 2.** Create a GitHub Actions workflow that runs the tests on each push

[ ] **Step 3.** Run the evaluation suite and record final metrics in the README

[ ] **Step 4.** Write the README sections and add the architecture diagram and screenshots

[ ] **Step 5.** Write a one-page business case with cost, benefit, risks and pricing idea

[ ] **Step 6.** Collect tester feedback, fix the top five issues and re-run all tests

Debugging log: errors I hit today

Error message	My guess	What fixed it

Code notes (what each part does, in my own words)

Day 18 review

Check your understanding

Answer in your own words, then compare with your notes and code.

Q1. Name four kinds of testing in this project.

Q2. What does CI do?

Q3. What belongs in the limits section?

Q4. What should a business case compare?

Q5. Why re-run the evaluation after fixes?

Words I learned today (write the meaning in your own words)

Word	My meaning
CI	
Load test	
Model card	
Business case	

Reflection

Today I am proud that:

The hardest part was:

One thing I will use this week:

Homework / challenge

Fix top 5 issues from testers.

Done? [ ] How it went:

Day 19: Pitch and portfolio

Objectives

[ ] Tell the story of your project for clients, employers and investors

[ ] Create a five-minute pitch with a live demo

[ ] Build a professional GitHub profile and portfolio

[ ] Update your CV with measurable results

Warm-up: one thing I tried since last class

Lab steps: tick each step and write notes

[ ] **Step 1.** Write the project story in five sentences: user, problem, change, result, limit

[ ] **Step 2.** Build a five-minute pitch with a diagram, one key number and a live demo path

[ ] **Step 3.** Record a backup demo video

[ ] **Step 4.** Clean up your GitHub: pin repos, write the profile README and remove secrets

[ ] **Step 5.** Write a one-page case study for your capstone and add it to your portfolio

[ ] **Step 6.** Rehearse the pitch twice with timing and collect feedback on clarity

Debugging log: errors I hit today

Error message	My guess	What fixed it

Code notes (what each part does, in my own words)

Day 19 review

Check your understanding

Answer in your own words, then compare with your notes and code.

Q1. What are the three parts of a story?

Q2. How long should the demo part of a pitch be?

Q3. Why record a backup demo?

Q4. What belongs on a case study?

Q5. What makes a CV line strong?

Words I learned today (write the meaning in your own words)

Word	My meaning
Pitch	
Case study	
Portfolio	
Backup	

Reflection

Today I am proud that:

The hardest part was:

One thing I will use this week:

Homework / challenge

Publish portfolio and case study.

Done? [ ] How it went:

Day 20: Demo Day and graduation

Objectives

[ ] Deliver a clear ten-minute demo with technical Q and A

[ ] Answer hard questions honestly

[ ] Give and receive structured technical feedback

[ ] Plan your next steps in career, freelancing or study

Warm-up: one thing I tried since last class

Lab steps: tick each step and write notes

[ ] **Step 1.** Do a full technical rehearsal including backup demo and internet check

[ ] **Step 2.** Deliver your ten-minute demo to the audience

[ ] **Step 3.** Answer at least three technical questions honestly

[ ] **Step 4.** Score two other teams with the six-criteria scorecard and give spoken feedback

[ ] **Step 5.** Complete your own reflection and the course feedback form

[ ] **Step 6.** Write your 90-day plan with three goals and a weekly build habit

Debugging log: errors I hit today

Error message	My guess	What fixed it

Code notes (what each part does, in my own words)

Day 20 review

Check your understanding

Answer in your own words, then compare with your notes and code.

Q1. How should a demo start?

Q2. What do you say if you do not know an answer?

Q3. Name three criteria on the scorecard.

Q4. What should you do with feedback?

Q5. What skill lasts longest in AI?

Words I learned today (write the meaning in your own words)

Word	My meaning
Demo Day	
Scorecard	
Open source	
Reflection	

Reflection

Today I am proud that:

The hardest part was:

One thing I will use this week:

Homework / challenge

Open-source your work and keep building.

Done? [ ] How it went:

Capstone plan

Capstone canvas

Question	My answer
Who is the user?	
What problem do they have?	
Architecture (tools, data, model)	
Evaluation plan	
Top 3 risks and defences	
Deployment and cost plan	

Snippet notebook

Write down code patterns and prompts you want to reuse.

Name	Snippet or prompt	Works? (1-5)

Feedback log

Record feedback from classmates and mentors.

From	Strength	Question	Idea

Course reflection and next steps

What I can do now that I could not do before:

My proudest piece of work and why:

What I will do in the next 30 days:

How I will help someone else learn:

Feedback for the course and trainers: